Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Programmation Parallèle — TP4 — Hands-on CUDA – Réduction (somme) et Produit matriciel sur GPU
4e année Cybersécurité - École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
KABORE Mohammad Sharif Jonathan

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. BAKHOUYA

Date : 1er novembre 2026

Table des matières

Rappel des objectifs du TP **2**

Rappels : réduction et produit matriciel sur GPU **2**

Architecture du projet **3**

Protocole de mesure et validation **3**
Complexité théorique 4

1 Exercice 2 : Réduction (somme) **5**
1.1 Principe 5
1.2 Résultats 6
1.3 Analyse complémentaire 9

2 Exercice 3 : Produit matrice-matrice **10**
2.1 Principe 10
2.2 Résultats 11
2.3 Analyse complémentaire 14

3 Synthèse et interprétation **15**

Conclusion **16**

Rappel des objectifs du TP

Ce second lab prolonge la découverte de CUDA : il s'agit de **porter sur GPU deux algorithmes classiques**, la réduction (somme des éléments d'un vecteur) et le produit matrice-matrice, afin d'exploiter le parallélisme massif de la carte, puis d'**analyser l'influence de la taille des données, des accès mémoire et de la configuration de la grille** sur les performances. Deux exercices sont réalisés :

- **Exercice 2 : Réduction (somme)** — portage de `sum.c` sur GPU (`sum.cu` et version modulaire) ; trois configurations de lancement comparées : `<<<N, 1>>>`, `<<<1, N>>>` et `<<<N/T, T>>>`
- **Exercice 3 : Produit matrice-matrice** $a = b \times c$ — portage de `mmult.c` sur GPU en complétant le gabarit `mmult.cu` ; mêmes trois configurations, sur des matrices $N \times N$

Pour chaque exercice, les deux algorithmes sont implémentés sur CPU et sur GPU, les temps d'exécution sont mesurés pour plusieurs tailles de données, l'accélération $S = T_{cpu}/T_{gpu}$ est calculée, et l'influence de la taille des données et des accès mémoire est analysée.

Rappels : réduction et produit matriciel sur GPU

Le modèle d'exécution (hôte et *device*, grille de blocs de threads, *warps* de 32 threads, lancement asynchrone chronométré avec des événements CUDA) est celui du Lab 1. Deux notions supplémentaires sont nécessaires ici.

Réduction. Contrairement à l'addition de vecteurs, tous les éléments contribuent à *un seul* résultat : les threads ne sont plus indépendants et doivent **coopérer**. La méthode retenue comporte deux niveaux :

1. **dans un bloc** : chaque thread charge un élément dans la **mémoire partagée** (`__shared__`, rapide et visible par tous les threads du bloc), puis les sommes partielles sont combinées par un **arbre** : à chaque étape, la moitié des threads encore actifs ajoute à sa valeur celle d'un voisin situé à distance s, avec $s = T/2,\ T/4,\ \ldots,\ 1$. Une barrière `__syncthreads()` sépare chaque étape ; la réduction d'un bloc de T threads demande $\lceil \log_2 T \rceil$ étapes ;
2. **entre blocs** : les blocs ne peuvent pas se synchroniser entre eux pendant un noyau ; le thread 0 de chaque bloc ajoute donc sa somme partielle au résultat global avec une opération **atomique** (`atomicAdd` sur `double`).

L'addition flottante n'étant pas associative, l'ordre dans lequel les blocs ajoutent leur contribution (non déterministe) change légèrement le résultat par rapport à la somme séquentielle du CPU : la vérification se fait donc avec une **tolérance relative** et non par égalité stricte.

Produit matriciel. Les matrices $N \times N$ de `float` sont stockées dans un tableau à une dimension, l'élément (i, j) étant en `a[i + j*N]`. Le produit s'écrit a_{ij} += $\sum_k b_{ik}\, c_{kj}$: N^2 éléments à calculer, chacun demandant N multiplications-additions, soit $2N^3$ opérations flottantes au total pour seulement $3N^2$ valeurs. Contrairement à l'addition de vecteurs, l'**intensité arithmétique croît avec** N : le produit matriciel est le cas typique où le GPU peut être rentable.

Architecture du projet

Les codes fournis (`sum.c`, `mmult.c`, `mmult.cu`) sont conservés, et le travail est organisé comme au Lab 1 :

- `Ex2/somme.cu / somme.cuh` : noyau `kernel_sum`, `atomic_add_double`, référence CPU `sum_cpu` et les trois fonctions GPU (une par configuration)
- `Ex3/matrice.cu / matrice.cuh` : noyau `kernel_mmult`, `minit`, référence CPU `mmult_cpu` et les trois fonctions GPU
- `benchmark.cu / benchmark.cuh` (un par exercice) : mesures, vérification du résultat, écriture du CSV
- `main.cu` (un par exercice) : lecture de N (argument), contrôle de la divisibilité par T, appel du benchmark
- `Makefile` (un par exercice) : compilation (`make`), mesures (`make bench`) et graphes (`make graphes`)
- `graphes.py` : génération des 3 graphes (`matplotlib`) à partir de `resultats.csv`
- `resultats.csv` : une ligne par taille et par configuration (colonnes `taille`, `config`, `t_cpu`, `t_gpu`)
- `Ex2/sum.cu` et `Ex3/mmult.cu` : versions simples, sur le modèle de `vecadd.cu` au Lab 1 (`mmult.cu` est le gabarit de l'énoncé dont le noyau est complété)
- `Ex2/sum.c`, `Ex3/mmult.c` : versions CPU de l'énoncé, conservées telles quelles
- `bin/` : exécutables produits par les Makefiles (`ex2_sum`, `ex3_mmult`)

T (`T_BLOCK`) est une constante de compilation, fixée par le Makefile : 512 pour la somme (`-DT_BLOCK=512`) et 128 pour le produit matriciel (`-DT_BLOCK=128`). Le noyau de la somme teste la borne (`i < N`), celui du produit parcourt la matrice par des boucles à pas de grille ; `main.cu` exige néanmoins que N soit un multiple de T. Les graphes sont produits avec `python3 graphes.py resultats.csv graphes`.

Protocole de mesure et validation

- **GPU utilisé :** NVIDIA GeForce RTX 3060 Laptop GPU (relevé avec `nvidia-smi`), *compute capability* **8.6** (architecture Ampere), **6 144 Mio** de mémoire, pilote **580.178.04**, version CUDA maximale supportée par le pilote : **13.0**. Compilation avec `nvcc -O2`. Les mesures sont celles de la même machine que pour le Lab 1.
- **Exercice 2, tailles testées :** $N \in \{512,\ 1024,\ 65\,536,\ 262\,144,\ 1\,048\,576,\ 4\,194\,304,\ 16\,777\,216\}$ éléments de type `double`, multiples de $T = 512$; une exécution supplémentaire avec la taille par défaut du code ($N = 2\,097\,152$) est faite par `make run`. `<<<1, N>>>` n'est testée que pour $N \leq 1024$.
- **Exercice 3, tailles testées :** matrices $N \times N$ avec $N \in \{128,\ 256,\ 512,\ 1024,\ 2048\}$ (de type `float`), multiples de $T = 128$; `make run` utilise $N = 1024$. Le cas $N = 4096$ du gabarit n'a pas été retenu : il demande $1{,}4 \cdot 10^{11}$ opérations, soit 8 fois le travail de $N = 2048$ (qui prend déjà 4,7 s sur le CPU), donc de l'ordre de 40 s ou plus par mesure CPU. `<<<1, N>>>` n'est testée que pour $N \leq 1024$.

— T_{cpu} : temps de la boucle séquentielle, mesuré avec `clock_gettime` (`CLOCK_MONOTONIC`, résolution de l'ordre de la nanoseconde, ce qui évite la limite de la microseconde de `gettimeofday` rencontrée au Lab 1). Moyenne sur **5 répétitions** pour la somme ; pour le produit, **3 répétitions** si $N \leq 512$ et **une seule** au-delà (durée trop longue).
— T_{gpu} : temps du **noyau seul**, mesuré avec des événements CUDA (`cudaEventRecord`, `cudaEventSynchronize`, `cudaEventElapsedTime`), moyenne sur **5 répétitions** (somme) ou **3 répétitions** (produit). Les `cudaMalloc`, `cudaMemset` et `cudaMemcpy` sont **exclus** de la mesure.
— **Échauffement :** un appel `cudaFree(0)` en début de benchmark force l'initialisation du contexte CUDA avant la première mesure ; au Lab 1, le coût d'initialisation faussait le premier point (voir la section analyse).
— Accélération $S = T_{cpu}/T_{gpu}$; débit en GFLOPS : $(N-1)/T$ pour la somme ($N-1$ additions) et $2N^3/T$ pour le produit (2 opérations par itération).
— **Vérification :** après chaque configuration, le résultat GPU est comparé à la référence CPU. Somme : erreur relative inférieure à 10^{-9} (l'ordre d'addition différant). Produit : comparaison élément par élément ; avec $b = c = 1$ et $a = 0$, chaque élément doit valoir exactement N, représentable exactement en `float` pour ces tailles.
— Chaque répétition GPU refait le cycle complet allocation → copie vers le *device* → noyau chronométré → copie vers l'hôte → libération ; seul le noyau est chronométré.
— Les configurations sont mesurées dans l'ordre `<<<N,1>>>`, `<<<N/T,T>>>` puis `<<<1,N>>>`, dans un même processus par taille ; la référence CPU est mesurée avant.

Complexité théorique

Opération	**Calcul séq.**	**Calcul / thread**	**Threads utiles**	**Données**	**Calcul / donnée**
Somme de N `double`	$O(N)$	$O(1)$ + arbre $O(\log T)$ par bloc	N	N doubles $= 8N$ octets	$O(1)$
Produit $N \times N$	$O(N^3)$	$O(N)$	N^2	$3N^2$ flottants $= 12N^2$ octets	$O(N)$

Table 1 – Complexité des deux algorithmes

La somme effectue ≈ 1 addition par 8 octets lus : intensité arithmétique de 1/8 flop par octet, comme l'addition de vecteurs, donc une opération **limitée par la bande passante mémoire**. Le produit matriciel effectue $2N^3$ opérations sur $12N^2$ octets : l'intensité arithmétique vaut $N/6$ flop par octet si chaque donnée n'était transférée qu'une fois, et croît avec N. Dans l'implémentation naïve choisie (sans mémoire partagée), chaque itération relit pourtant b et c en mémoire globale (soit 2 opérations pour 8 octets, 1/4 flop par octet) : le gain dépend alors de la capacité des caches à absorber ces relectures.

1 Exercice 2 : Réduction (somme)

1.1 Principe

Le code CPU fourni (`sum.c`) additionne séquentiellement les N éléments d'un vecteur de `double` initialisé avec `rand()/RAND_MAX` (graine 1). Son port GPU utilise un **noyau unique** pour les trois configurations ; seul le lancement change. Chaque bloc réduit ses `blockDim.x` éléments en mémoire partagée, puis son thread 0 ajoute le résultat partiel à la somme globale :

```
__global__ void kernel_sum(const double *A, double *res, int N) {
    extern __shared__ double sdata[];
    int tid = threadIdx.x;
    int i   = blockIdx.x * blockDim.x + tid;
    int p, s;

    sdata[tid] = (i < N) ? A[i] : 0.0;
    __syncthreads();

    /* plus petite puissance de 2 >= blockDim.x */
    p = 1;
    while (p < (int)blockDim.x) p <<= 1;

    for (s = p >> 1; s > 0; s >>= 1) {
        if (tid < s && tid + s < (int)blockDim.x)
            sdata[tid] += sdata[tid + s];
        __syncthreads();
    }

    if (tid == 0)
        atomic_add_double(res, sdata[0]);
}
```

La taille de la mémoire partagée est donnée au lancement (troisième paramètre des chevrons), et le résultat global est mis à zéro avant le noyau (`cudaMemset`). `atomic_add_double` utilise `atomicAdd` natif à partir de l'architecture `sm_60`, et une boucle `atomicCAS` sinon :

```
__device__ static double atomic_add_double(double *addr, double val) {
#if defined(__CUDA_ARCH__) && __CUDA_ARCH__ >= 600
    return atomicAdd(addr, val);
#else
    unsigned long long *p = (unsigned long long *)addr;
    unsigned long long old = *p, assumed;
    do {
        assumed = old;
        old = atomicCAS(p, assumed,
                __double_as_longlong(val + __longlong_as_double(
                   assumed)));
    } while (assumed != old);
    return __longlong_as_double(old);
#endif
}
```

Les trois configurations se lancent ainsi (`launch_and_time` fixe aussi la taille de mémoire partagée à `threads * sizeof(double)`) :

```
t = launch_and_time(dA, dRes, N, N, 1);                    /* <<<N, 1>>>
      */
t = launch_and_time(dA, dRes, N, 1, N);                    /* <<<1, N>>>
      */
t = launch_and_time(dA, dRes, N, N / T_BLOCK, T_BLOCK);   /* <<<N/T, T
   >>> */
```

Lancement	**Blocs**	**Threads / bloc**	**Conséquence pour la réduction**
`<<<N, 1>>>`	N	1	aucune réduction dans le bloc ; N `atomicAdd` sur la **même** adresse mémoire, sérialisés
`<<<1, N>>>`	1	N	arbre de $\lceil \log_2 N \rceil$ étapes dans un seul bloc, donc un seul multiprocesseur ; un seul `atomicAdd` ; $N \leq 1024$
`<<<N/T, T>>>`	N/T	$T = 512$	9 étapes d'arbre par bloc, N/T `atomicAdd` seulement

TABLE 2 – Les trois configurations de lancement comparées pour la somme

Compilation et exécution :

```
make          # compile bin/ex2_sum  (nvcc -O2 -DT_BLOCK=512)
make bench    # une execution par taille -> resultats.csv
make graphes  # graphes/sum_temps.png, _speedup.png, _gflops.png
bin/ex2_sum 1048576 resultats.csv     # une seule taille
```

1.2 Résultats

N	T_{cpu} (μs)	`<<<N,1>>>` : T_{gpu} (μs) [S]	`<<<1,N>>>` : T_{gpu} (μs) [S]	`<<<N/T,T>>>` : T_{gpu} (μs) [S]
512	0,6	30,5 [0,02]	11,5 [0,05]	10,0 [0,06]
1024	1,2	30,7 [0,04]	14,2 [0,08]	9,7 [0,12]
65 536	97,1	157,9 [0,61]	—	15,0 [6,5]
262 144	308,8	601,3 [0,51]	—	41,2 [7,5]
1 048 576	1 092,5	2 265,1 [0,48]	—	137,6 [7,9]
4 194 304	4 907,6	9 032,5 [0,54]	—	511,8 [9,6]
16 777 216	18 871,2	30 486,1 [0,62]	—	1 765,2 [10,7]

TABLE 3 – Somme : temps moyens (5 répétitions, noyau seul) et accélération $S = T_{cpu}/T_{gpu}$

N	CPU	`N,1`	`1,N`	`N/T,T`	Débit mémoire `N/T,T` (Go/s)	Volume $8N$ (Mo)
512	0,92	0,017	0,045	0,051	0,4	0,004
1024	0,88	0,033	0,072	0,106	0,8	0,008
65 536	0,68	0,42	—	4,38	35,0	0,52
262 144	0,85	0,44	—	6,37	50,9	2,1
1 048 576	0,96	0,46	—	7,62	60,9	8,4
4 194 304	0,86	0,46	—	8,20	65,6	33,6
16 777 216	0,89	0,55	—	9,51	76,0	134,2

TABLE 4 – Somme : débit en GFLOPS ($(N-1)/T$), débit mémoire effectif de `<<<N/T,T>>>` (calculé avec $8N$ octets lus par noyau) et volume de données

N	Rapport $T_{N,1}/T_{N/T,T}$	`N,1` : ns par bloc	`N/T,T` : ns par élément	CPU : ns par élément
512	3,0	59,6	19,6	1,09
1024	3,2	30,0	9,44	1,14
65 536	10,6	2,41	0,228	1,48
262 144	14,6	2,29	0,157	1,18
1 048 576	16,5	2,16	0,131	1,04
4 194 304	17,6	2,15	0,122	1,17
16 777 216	17,3	1,82	0,105	1,13

TABLE 5 – Somme : coût comparé des configurations (temps divisé par le nombre de blocs ou d'éléments)

Exécution à la taille par défaut ($N = 2\,097\,152$, `make run`) : $T_{cpu} = 2\,610{,}6$ μs ; `<<<N,1>>>` 5 126,4 μs ($S = 0{,}51$) ; `<<<N/T,T>>>` 308,8 μs ($S = 8{,}45$). Ces valeurs s'insèrent dans la tendance du tableau 3, entre $N = 1\,048\,576$ et $N = 4\,194\,304$.

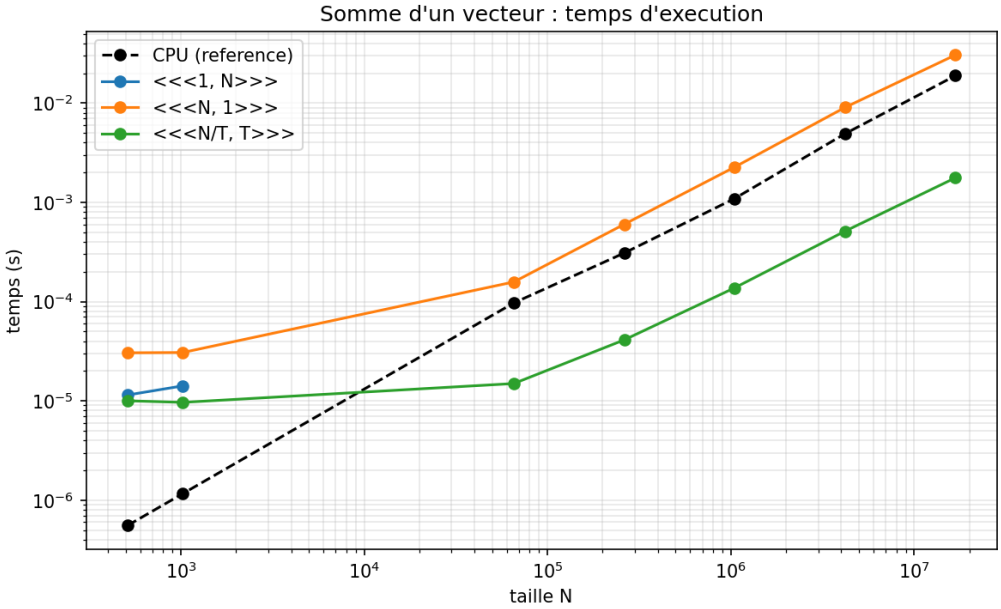


FIGURE 1 – Somme : temps d'exécution

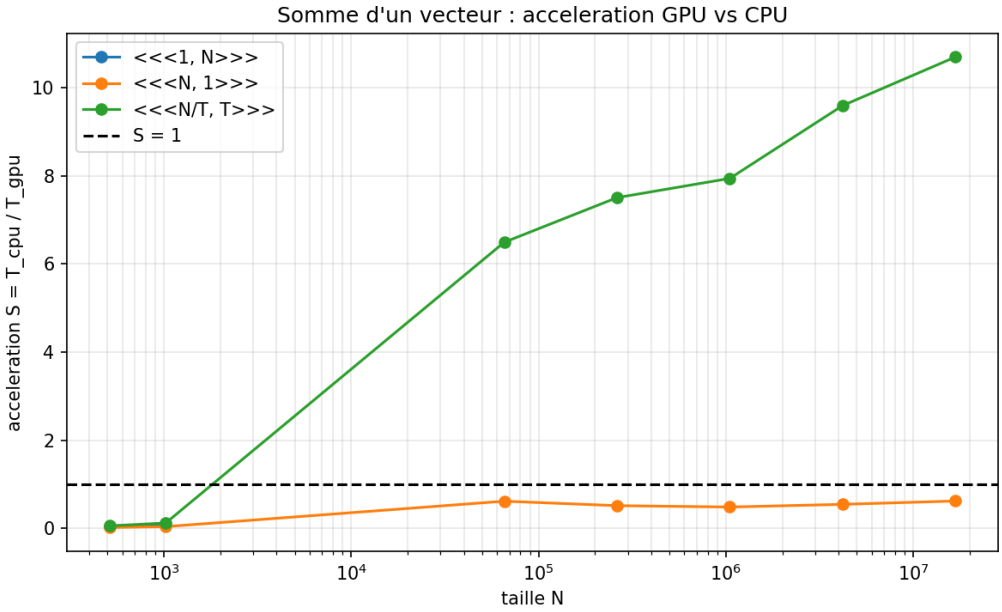


FIGURE 2 – Somme : accélération GPU vs CPU

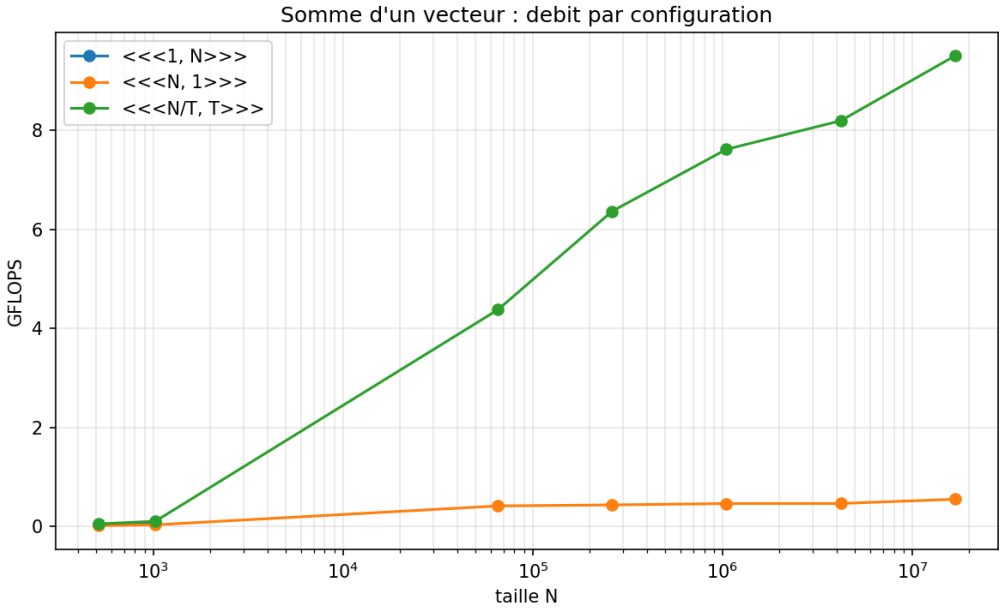


FIGURE 3 – Somme : débit (GFLOPS)

Observations :

— **`<<<N, 1>>>` est la pire configuration** : son accélération reste inférieure à 1 pour toutes les tailles (de 0,02 à 0,62) : le GPU est **moins rapide que le CPU** (figure 2). Elle est de 10 à 18 fois plus lente que `<<<N/T, T>>>` dès $N \geq 65\,536$.

— Pour $N \geq 65\,536$, son temps est proportionnel au nombre de blocs (pente log-log $\approx 0{,}95$; $\approx 1{,}8$ à $2{,}4$ ns par bloc, tableau 5) et son débit plafonne vers 0,4–0,55 GFLOPS (figure 3). Deux effets s'y cumulent : chaque bloc d'un seul thread n'occupe qu'une voie sur 32 de son *warp*, et les N `atomicAdd` visent la même adresse, donc s'exécutent l'un après l'autre. Les mesures ne permettent pas de séparer les deux contributions.

— **`<<<N/T, T>>>` devient rentable entre $N = 10^3$ et $N = 6{,}5 \cdot 10^4$** : l'accélération passe de 0,12 à 6,5, puis atteint 10,7 pour $N = 16\,777\,216$. Par interpolation, le croisement

avec le CPU se situe vers $N \approx 10^4$ (figure 1) ; il n'a pas été mesuré, aucune taille n'étant testée dans cet intervalle.

— Pour $N \leq 1024$, le GPU est toujours plus lent que le CPU ($S \leq 0{,}12$) : T_{gpu} reste de l'ordre de 10 à 14 μs pour `<<<N/T, T>>>` et `<<<1, N>>>`, alors que le CPU n'a besoin que de 0,6 à 1,2 μs. Ce plancher est supérieur d'environ 4 μs à celui du Lab 1 (6 à 9 μs), ce qui est cohérent avec un noyau plus long (réduction et `atomicAdd`). La configuration `<<<N,1>>>` se stabilise plus haut (≈ 30 μs).

— `<<<1, N>>>` n'est possible que pour $N \leq 1024$ et n'utilise qu'un seul multiprocesseur. Pour $N = 512$, `<<<N/T, T>>>` est *exactement* le lancement `<<<1, 512>>>` : l'écart mesuré (10,0 contre 11,5 μs) est du bruit de mesure. Pour $N = 1024$, `<<<N/T, T>>>` lance 2 blocs de 512 threads (arbre de 9 étapes, deux multiprocesseurs en parallèle) et est plus rapide (9,7 contre 14,2 μs) que le bloc unique de 1024 threads (arbre de 10 étapes).

— **Le débit de `<<<N/T, T>>>` n'est pas saturé** : il passe de 35 à 76 Go/s de mémoire lue et le temps par élément diminue encore entre $N = 4\,194\,304$ (0,122 ns) et $N = 16\,777\,216$ (0,105 ns) ; la pente log-log n'est que de 0,87 sur $N \geq 65\,536$. Le coût fixe de lancement continue donc à s'amortir, et des vecteurs encore plus grands (134 Mo à $N = 16\,777\,216$) donneraient une accélération supérieure à 10,7.

— **Débit CPU constant** : le CPU reste à environ 0,9 GFLOPS (de 1,0 à 1,5 ns par élément) pour toutes les tailles, sans effet de cache visible : la boucle `s += A[i]` est une chaîne de dépendances sur `s`, et le compilateur ne peut pas la vectoriser sans réassocier les additions flottantes (ce que `-O2` n'autorise pas). Cette explication est plausible mais n'a pas été vérifiée (par exemple avec une version à plusieurs accumulateurs).

— **Pas de point aberrant** à $N = 512$: contrairement au Lab 1 (3,3 ms pour `<<<N,1>>>`), la première mesure n'est pas faussée (30,5 μs). L'appel `cudaFree(0)` ajouté avant les mesures est cohérent avec l'explication donnée au Lab 1 (initialisation du premier lancement), sans que cela soit démontré.

1.3 Analyse complémentaire

Pourquoi `<<<N, 1>>>` est-elle si lente ? Comme pour l'addition du Lab 1 (où elle atteignait $\approx 1{,}5$ ns par bloc), le temps est proportionnel au nombre de blocs, avec un coût un peu plus élevé ici (≈ 2 ns par bloc) : chaque bloc exécute en plus un `atomicAdd` sur une adresse partagée par tous les autres. Sur 16 777 216 blocs, cela fait 16 777 216 opérations atomiques sérialisées. À l'inverse, `<<<N/T, T>>>` n'en émet que $N/T = 32\,768$.

Pourquoi le débit de `<<<N/T, T>>>` reste-t-il modeste ? Le Lab 1 avait observé 215 à 237 Go/s pour l'addition de vecteurs avec la même configuration ; la somme n'atteint ici que 76 Go/s. Plusieurs causes sont plausibles, sans qu'elles aient été isolées par des mesures : (i) l'arbre de réduction n'est pas une opération purement mémoire : il enchaîne 9 barrières `__syncthreads()` où la moitié, puis le quart, etc. des threads du bloc sont inactifs ; (ii) chaque thread ne charge qu'un seul élément avant de commencer à réduire, ce qui amortit mal le coût de l'arbre ; (iii) les N/T `atomicAdd` sur la même adresse s'ajoutent à la fin de chaque bloc ; (iv) la précision double est lue sur 8 octets par élément, et les GPU grand public exécutent la double précision beaucoup plus lentement que la simple, ce qui compte peu pour une opération limitée par la mémoire mais pas nul pour l'arbre. Une version où chaque thread additionne plusieurs éléments avant la réduction en mémoire partagée (boucle à pas de grille) réduirait (i), (ii) et (iii) ; elle n'a pas été testée.

Influence de la taille N (`<<<N/T, T>>>`) : trois régimes apparaissent.

— *Régime de latence* ($N \leq 1024$, 1 à 2 blocs) : $T_{gpu} \approx 10$ μs, coût fixe de lancement ; le CPU termine en moins de 1,2 μs, aucun gain n'est possible.

— *Transition* ($65\,536 \leq N \leq 262\,144$) : le GPU est 6,5 à 7,5 fois plus rapide, mais le débit mémoire (35 à 51 Go/s) reste loin de la valeur atteinte aux grandes tailles : pour $N = 65\,536$, les 15 μs comprennent encore une grande part de coût fixe.
— *Régime de débit* ($N \geq 1\,048\,576$) : T_{gpu} devient presque proportionnel à N (de 1 048 576 à 16 777 216, les données sont multipliées par 16 et le temps par 12,8), le débit mémoire monte lentement de 61 à 76 Go/s et l'accélération atteint 7,9 puis 10,7.

Place des transferts. Comme au Lab 1, l'accélération est celle du **noyau seul**. La somme échange $8N$ octets avec le *device* pour $N - 1$ additions, soit 134 Mo pour $N = 16\,777\,216$; si le lien PCIe soutenait de l'ordre de 10 Go/s (ordre de grandeur supposé, non mesuré), cette seule copie durerait ≈ 13 ms, contre 1,8 ms pour le noyau. Le total (≈ 15 ms) resterait sous le temps du CPU (18,9 ms), mais l'accélération tomberait de 10,7 à $\approx 1{,}3$. Une réduction isolée ne justifie donc guère le GPU si les données doivent y être copiées pour cette seule opération ; elle est surtout utile lorsque les données y résident déjà (en fin d'un calcul GPU, par exemple).

Fiabilité des mesures. L'exécution à la taille par défaut ($N = 2\,097\,152$) s'insère entre les points voisins du tableau. Les temps sont globalement monotones, y compris pour le CPU (1,0 à 1,5 ns par élément). Aucun point aberrant n'apparaît, mais cinq répétitions sans lancement d'échauffement séparé restent peu nombreuses.

2 Exercice 3 : Produit matrice-matrice

2.1 Principe

Le code CPU fourni (`mmult.c`) calcule $a = b \times c$ avec trois boucles imbriquées (ordre j, k, i, la boucle interne parcourant les éléments contigus de a et b) sur des matrices initialisées par `minit` ($a = 0$, $b = c = 1$) : chaque élément du résultat vaut donc N, ce qui permet de vérifier le résultat. Le gabarit `mmult.cu` lance une grille `dim3 grid(N/T, N)` de blocs de T threads ; son noyau est vide dans l'énoncé. Dans notre version, **un thread calcule un élément** $a[i + j \cdot N]$:

```
__global__ void kernel_mmult(float *a, float *b, float *c, int N) {
    int i, j, k;
    for (j = blockIdx.y; j < N; j += gridDim.y)
        for (i = blockIdx.x * blockDim.x + threadIdx.x; i < N;
             i += gridDim.x * blockDim.x) {
            float s = a[i + j * N];
            for (k = 0; k < N; k++)
                s += b[i + k * N] * c[k + j * N];
            a[i + j * N] = s;
        }
}
```

Les boucles à pas de grille (*grid-stride*) font que le **même noyau** couvre toute la matrice quelle que soit la grille : seul le lancement change, comme au Lab 1. L'accumulation se fait dans un registre (`s`) avec une seule écriture finale en mémoire globale.

```
launch_and_time(dA, dB, dC, N, dim3(N, N),       dim3(1));        /*
   "<<<N, 1>>>"   */
launch_and_time(dA, dB, dC, N, dim3(1, 1),       dim3(N));        /*
   "<<<1, N>>>"   */
launch_and_time(dA, dB, dC, N, dim3(N / T_BLOCK, N), dim3(T_BLOCK));
   /* "<<<N/T, T>>>" */
```

Config.	Grille	Threads / bloc	Travail par thread	Remarque
`<<<N, 1>>>`	(N, N)	1	1 élément (N itérations)	N^2 blocs d'un thread ; 1 voie sur 32 active par warp
`<<<1, N>>>`	$(1, 1)$	N	N éléments (N^2 itérations)	un seul bloc, un seul multiprocesseur ; $N \leq 1024$
`<<<N/T, T>>>`	$(N/T, N)$	$T = 128$	1 élément (N itérations)	grille du gabarit : blocs nombreux et remplis de threads

TABLE 6 – Les trois configurations pour le produit matriciel (les notations `<<<N,1>>>` et `<<<1,N>>>` désignent ici une grille à deux dimensions, N étant la dimension de la matrice)

Compilation et exécution :

```
make           # compile bin/ex3_mmult  (nvcc -O2 -DT_BLOCK=128)
make bench     # N = 128 ... 2048 -> resultats.csv
make graphes   # graphes/mmult_temps.png, _speedup.png, _gflops.png
bin/ex3_mmult 1024 resultats.csv     # une seule taille
```

2.2 Résultats

N	T_{cpu} (ms)	`<<<N,1>>>` : T_{gpu} (ms) [S]	`<<<1,N>>>` : T_{gpu} (ms) [S]	`<<<N/T,T>>>` : T_{gpu} (ms) [S]
128	1,0	0,271 [3,7]	0,449 [2,2]	0,0133 [74,8]
256	7,49	1,761 [4,3]	2,536 [3,0]	0,0628 [119,3]
512	62,6	13,75 [4,6]	15,65 [4,0]	0,451 [138,9]
1024	501,8	170,0 [3,0]	83,3 [6,0]	4,69 [107,0]
2048	4 721,6	1 385,2 [3,4]	—	121,7 [38,8]

TABLE 7 – Produit matriciel : temps moyens (noyau seul) et accélération $S = T_{cpu}/T_{gpu}$

N	CPU	`N,1`	`1,N`	`N/T,T`	Volume $12N^2$ (Mo)
128	4,21	15,5	9,3	314,6	0,20
256	4,48	19,1	13,2	534,3	0,79
512	4,29	19,5	17,2	595,8	3,1
1024	4,28	12,6	25,8	457,8	12,6
2048	3,64	12,4	—	141,2	50,3

TABLE 8 – Produit matriciel : débit en GFLOPS ($2N^3/T$) et volume de données

N	Rapport $T_{N,1}/T_{N/T,T}$	`N,1` : ns par bloc	`N,1` : ns par itération et bloc	`N/T,T` : ns par élément	Taille de b (Mio)
128	20,3	16,5	0,129	0,81	0,06
256	28,0	26,9	0,105	0,96	0,25
512	30,5	52,4	0,102	1,72	1,0
1024	36,2	162	0,158	4,47	4,0
2048	11,4	330	0,161	29,0	16,0

TABLE 9 – Produit matriciel : coût comparé des configurations (« itération » : une multiplication-addition de la boucle sur k)

Exécution à $N = 1024$ (`make run`), seconde mesure indépendante : $T_{cpu} = 513{,}9$ ms ; `<<<N,1>>>` 179,1 ms ; `<<<1,N>>>` 82,6 ms ; `<<<N/T,T>>>` 4,73 ms. Les écarts avec le tableau 7 sont de +2,4 % (CPU), +5,4 % (`N,1`), −0,8 % (`1,N`) et +0,8 % (`N/T,T`) : la mesure est reproductible à quelques pour cent près pour cette taille.

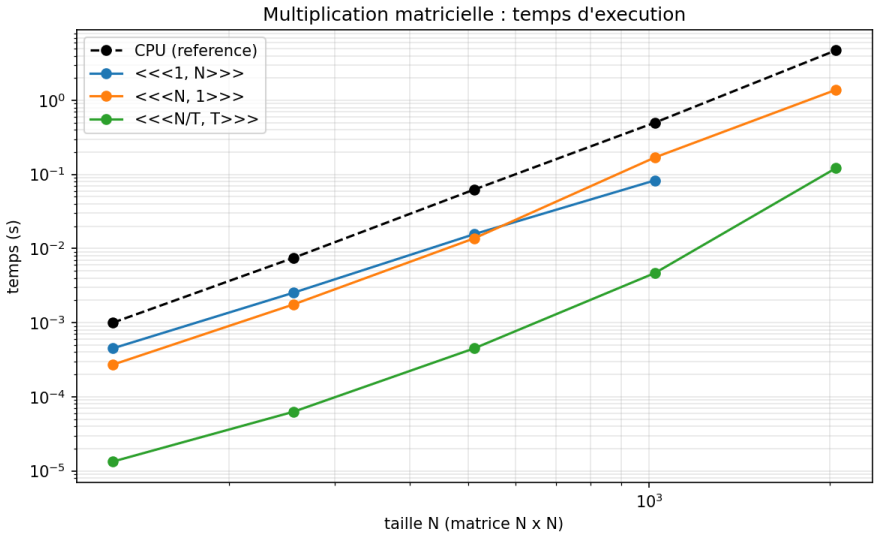


FIGURE 4 – Produit matriciel : temps d'exécution

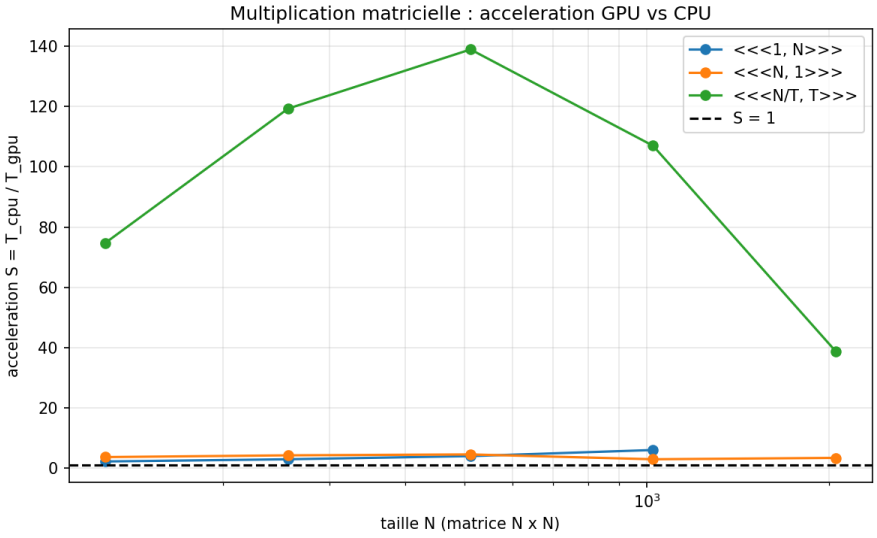


FIGURE 5 – Produit matriciel : accélération GPU vs CPU

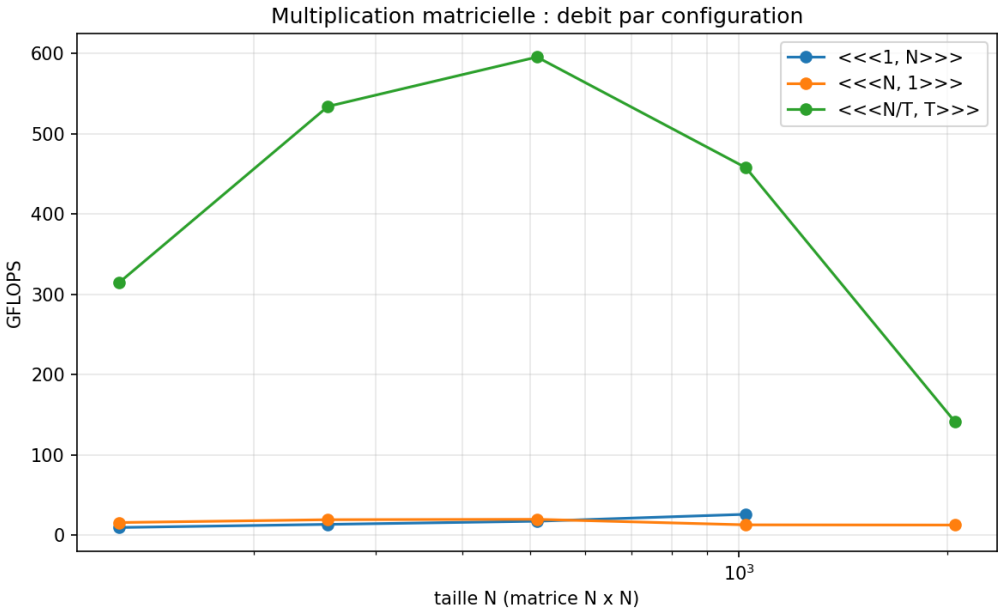


FIGURE 6 – Produit matriciel : débit (GFLOPS)

Observations :

— **`<<<N/T, T>>>` est de loin la meilleure configuration** pour toutes les tailles : accélération de 74,8 à 138,9 pour $128 \leq N \leq 512$, 107 à $N = 1024$ et 38,8 à $N = 2048$ (figure 5). Le débit atteint 596 GFLOPS à $N = 512$ contre ≈ 4 GFLOPS pour le CPU. Contrairement à l'addition et à la somme, le GPU est déjà 75 fois plus rapide pour la plus petite taille testée : le calcul ($2N^3$ opérations) est assez important pour que le coût fixe de lancement (≈ 6 à 10 μs au Lab 1 et pour la somme) ne suffise pas à effacer le gain.

— **`<<<N, 1>>>` bat ici le CPU**, ce qui n'arrivait jamais pour l'addition (Lab 1) ni pour la somme : accélération de 3,0 à 4,6, soit 12 à 19,5 GFLOPS contre ≈ 4 pour le CPU (figure 6). Le coût d'ordonnancement d'un bloc (≈ 2 ns, mesuré au Lab 1 et pour la somme) devient négligeable quand chaque bloc effectue N itérations (0,10 à 0,16 ns par itération, tableau 9), donc 16 à 330 ns par bloc. Elle reste 11 à 36 fois plus lente que `<<<N/T, T>>>`.

— `<<<1, N>>>` (un seul bloc, un seul multiprocesseur) est la pire des trois pour $N \leq 512$ (2,2 à 4,0 de speedup), mais **dépasse `<<<N,1>>>` pour** $N = 1024$ (83,3 contre 170 ms, $S = 6{,}0$ contre 3,0) : son débit croît avec N (9,3 puis 13,2, 17,2 et 25,8 GFLOPS), alors que celui de `<<<N,1>>>` décroît au-delà de $N = 512$. Une explication plausible, non vérifiée par des mesures de profilage : un bloc de N threads contient $N/32$ *warps* (4 à 32) qui, résidant sur le même multiprocesseur, masquent mieux la latence mémoire à mesure que N augmente.

— **Le débit de `<<<N/T, T>>>` s'effondre pour les grandes matrices** : il passe de 596 GFLOPS ($N = 512$) à 458 ($N = 1024$) puis 141 ($N = 2048$). Entre $N = 1024$ et $N = 2048$, le travail est multiplié par 8 mais le temps par 25,9 (4,69 puis 121,7 ms) ; pour le CPU, le même saut ne multiplie le temps que par 9,4 (tableau 7). Ce n'est donc pas un simple effet de la complexité en $O(N^3)$: l'efficacité du noyau GPU chute d'un facteur $\approx 3{,}2$ (voir l'analyse complémentaire). La figure 5 montre la même chute, après un maximum à $N = 512$.

— **Débit CPU quasi constant** : 4,2 à 4,5 GFLOPS pour $N \leq 1024$, 3,6 GFLOPS à $N = 2048$ (les trois matrices occupent alors 48 Mio). Le temps CPU est multiplié par

7,5 à 9,4 à chaque doublement de N (théorie : 8), pente log-log $\approx 3{,}05$.

— **Mesure unique pour la grande taille** : le CPU à $N = 1024$ et $N = 2048$ n'est mesuré qu'une fois (voir le protocole), et la figure 5 contient donc, pour ces deux points, une incertitude supplémentaire. La seconde exécution à $N = 1024$ donne 513,9 ms pour le CPU, soit 2,4 % d'écart.

2.3 Analyse complémentaire

Accès mémoire du noyau. Dans l'itération k du noyau, les T threads d'un bloc (valeurs consécutives de i, même j) lisent `b[i + k*N]` à des adresses **consécutives** (accès coalescé : un seul transfert mémoire regroupe un *warp* de 32 threads) et `c[k + j*N]` à une **même** adresse (diffusion à tout le *warp*). Chaque élément de a est écrit une seule fois. Ce schéma est favorable. Il reste que tous les blocs d'une même position x (donc les N blocs de $j = 0, \ldots, N-1$) relisent la matrice b entière : le noyau n'a pas de réutilisation explicite des données en mémoire partagée et compte sur les caches du GPU.

Pourquoi le débit s'effondre-t-il à $N = 2048$? L'hypothèse la plus cohérente avec les mesures est celle de la **taille du jeu de données relu** : la matrice b occupe 0,25 Mio à $N = 256$, 1 Mio à $N = 512$, 4 Mio à $N = 1024$ et 16 Mio à $N = 2048$ (tableau 9). Tant que b tient dans le cache de niveau 2 du GPU, les relectures par les N blocs successifs sont servies par le cache ; au-delà, elles se font en mémoire globale. Le cache L2 de cette carte est de l'ordre de quelques Mio (valeur à confirmer avec `deviceQuery`), ce qui correspond à la transition observée entre $N = 512$ (596 GFLOPS) et $N = 1024$ (458), puis $N = 2048$ (141). Le temps par élément passe de 0,81 ns ($N = 128$) à 29 ns ($N = 2048$) alors que la quantité de calcul par élément n'augmente que d'un facteur 16 (N itérations), contre 36 pour le temps. Cette hypothèse n'a pas été testée (par exemple avec un profileur ou en changeant l'ordre de parcours des blocs) ; elle suggère qu'une version avec **mémoire partagée par tuiles** (chaque bloc charge une tuile de b et de c puis la réutilise) serait la première amélioration à essayer.

Pourquoi $N = 128$ est-il en dessous du maximum ? Pour $N = 128$ ($T = 128$), la grille `<<<N/T, T>>>` ne contient que $1 \times 128 = 128$ blocs de 128 threads, soit 16 384 threads au total, et le noyau dure 13,3 μs : le débit (315 GFLOPS) est limité par le faible nombre de threads et par le coût fixe de lancement, d'où la progression de $N = 128$ à $N = 512$ (315, 534 puis 596 GFLOPS) avant la chute des grandes tailles.

Part du pic théorique. Les 596 GFLOPS du meilleur point ne représentent qu'une petite fraction du pic en simple précision d'une carte de cette gamme (de l'ordre de la dizaine de TFLOPS selon les données constructeur, non vérifiées ici) : le noyau naïf est loin d'exploiter tout ce que la carte peut faire, ce qui confirme qu'il est limité par les accès mémoire plutôt que par le calcul.

Passage à l'échelle. Pente log-log du temps sur $128 \leq N \leq 2048$: $\approx 3{,}05$ pour le CPU (théorie : 3), $\approx 3{,}12$ pour `<<<N, 1>>>` et $\approx 3{,}25$ pour `<<<N/T, T>>>`. Pour ce dernier, la pente supérieure à 3 traduit la perte d'efficacité aux grandes tailles ; les temps croissent par facteurs de 4,7, 7,2, 10,4 puis 25,9 à chaque doublement de N (théorie : 8), preuve d'un régime de latence puis de saturation mémoire.

Place des transferts. Le produit échange $12N^2$ octets pour $2N^3$ opérations : 50,3 Mo à $N = 2048$. Si le lien PCIe soutenait de l'ordre de 10 Go/s (ordre de grandeur supposé, non mesuré), ces transferts dureraient ≈ 5 ms, soit ≈ 4 % du temps du noyau (121,7 ms) et très peu devant les 4,7 s du CPU. Contrairement à l'addition et à la somme, **le gain du GPU subsiste donc en tenant compte des transferts** pour les tailles testées. À $N = 128$, ces transferts (0,20 Mo, ≈ 20 μs à ce débit supposé) seraient du même ordre que le noyau (13,3 μs),

mais restent très inférieurs au temps CPU (1,0 ms).

Fiabilité des mesures.

— Les temps sont monotones et leur évolution est régulière, sauf la chute d'efficacité décrite plus haut, très marquée pour `<<<N/T, T>>>` et plus modeste pour `<<<N, 1>>>` (le temps par itération passe de 0,10 à 0,16 ns entre $N = 512$ et $N = 1024$), ce qui est cohérent avec l'hypothèse de saturation mémoire ; `<<<1, N>>>` ne la montre pas.
— La reproductibilité à $N = 1024$ est de 0,8 à 5,4 % selon la configuration (seconde exécution par `make run`).
— Pour $N = 1024$ et $N = 2048$, le CPU n'est mesuré qu'une fois et le GPU que trois fois ; les accélérations de ces deux points sont donc moins précises que celles des petites tailles.
— À $N = 128$, $T_{cpu} \approx 1$ ms reste mesurable avec `clock_gettime` ; le GPU (13,3 μs) est proche du plancher de lancement observé ($\approx$ 6 à 10 μs), et l'accélération de 74,8 est donc sensible à ce coût fixe.

3 Synthèse et interprétation

Configuration	**Parallélisme exploité**	S **max (somme)**	S **max (produit)**	**Verdict**
`<<<N, 1>>>`	N blocs d'un thread (1 voie active sur 32)	0,62	4,6	Jamais rentable pour la somme ; peu rentable pour le produit, mais meilleur que le CPU car chaque bloc fait N itérations
`<<<1, N>>>`	1 bloc de N threads, un seul multiprocesseur	0,08	6,0	Limitée à $N \leq 1024$; devient meilleure que `<<<N,1>>>` pour le produit à $N = 1024$
`<<<N/T, T>>>`	N/T blocs de T threads	10,7	138,9	Meilleure configuration partout ; rentable pour la somme dès $N \approx 10^4$ et pour le produit dès $N = 128$

TABLE 10 – Synthèse des trois configurations (valeurs relevées dans les tableaux)

La comparaison des deux exercices (et du Lab 1) met en évidence les points suivants :

— **Intensité arithmétique** : la somme (1/8 flop par octet, comme l'addition de vecteurs) est limitée par la mémoire et plafonne à 10,7 d'accélération et 9,5 GFLOPS ; le produit, où chaque donnée sert à $O(N)$ calculs, atteint 139 d'accélération et près de 600 GFLOPS. C'est la quantité de calcul par donnée qui décide du gain.
— **Coût d'ordonnancement des blocs** : `<<<N, 1>>>` coûte $\approx$ 2 ns par bloc. Ce coût domine pour les opérations légères (somme : $S < 1$) et devient secondaire quand chaque

bloc a beaucoup de travail (produit : $S = 3$ à 4,6).
— **Réduction** : elle introduit une coopération entre threads (arbre en mémoire partagée) puis entre blocs (`atomicAdd`) ; le débit de 76 Go/s reste inférieur à celui de l'addition de vecteurs du Lab 1 (215 à 237 Go/s).
— **Hiérarchie mémoire** : le noyau de produit, bon pour $N \leq 512$, perd un facteur ≈ 4 de débit quand la matrice relue ne tient plus dans les caches ; une version par tuiles en mémoire partagée est la piste d'amélioration naturelle.

Conclusion

Ce lab a permis de porter sur GPU la somme d'un vecteur (réduction par arbre en mémoire partagée et `atomicAdd`) et le produit matriciel, de les comparer à leur version CPU et d'analyser l'effet de la taille des données et de la configuration de la grille.

Résultats clés :
— La **configuration `<<<N/T, T>>>` est la meilleure** dans les deux exercices ; `<<<N, 1>>>` et `<<<1, N>>>` sont nettement plus lentes (11 à 36 fois pour le produit, jusqu'à 18 fois pour la somme).
— **Somme** : accélération de 6,5 ($N = 65\,536$) à 10,7 ($N = 16\,777\,216$) avec `<<<N/T, T>>>`, mais GPU plus lent que le CPU pour $N \leq 1024$ (coût de lancement ≈ 10 μs) ; `<<<N, 1>>>` ne dépasse jamais le CPU (accélération $\leq 0{,}62$).
— **Produit matriciel** : accélération maximale de 138,9 ($N = 512$, 596 GFLOPS) contre ≈ 4 GFLOPS pour le CPU ; elle retombe à 38,8 pour $N = 2048$ (débit de 141 GFLOPS), ce qui est attribué, par hypothèse, à la taille de la matrice relue par rapport aux caches. `<<<N, 1>>>` reste 3 à 4,6 fois plus rapide que le CPU et `<<<1, N>>>` atteint 6,0 à $N = 1024$.
— L'**intensité arithmétique** décide de l'intérêt du GPU : le produit matriciel y reste très rentable en tenant compte des transferts (estimation), alors que pour une somme isolée le gain ne serait plus que d'environ 1,3 (estimation).

Limites et pistes :
— Les accélérations portent sur le **noyau seul** ; les transferts hôte/*device* et les allocations sont exclus. Leur durée n'a pas été mesurée (le débit PCIe de 10 Go/s est un ordre de grandeur supposé).
— Les mesures reposent sur 5 répétitions (somme) ou 3 répétitions (produit), une seule pour le CPU à $N \geq 1024$; il serait préférable d'utiliser la médiane et de tester d'autres valeurs de T (64, 256, 512 pour le produit).
— Le Makefile ne précise pas d'architecture cible (`-arch`) : le noyau de la somme utilise `atomicAdd` natif sur `double` seulement si l'architecture compilée est au moins `sm_60`, sinon la version à boucle `atomicCAS`, plus lente sous contention. L'architecture réellement utilisée par `nvcc` n'a pas été vérifiée ; compiler avec `-arch=sm_86` (la *compute capability* de la carte) lèverait le doute.
— Les explications données pour la chute du débit du produit à $N = 2048$ (taille de la matrice relue et cache L2) et pour le débit modeste de la somme (arbre de réduction, un élément par thread) sont des **hypothèses** cohérentes avec les mesures, non confirmées par un profilage (`nsys`, `ncu`).
— Pistes d'amélioration : produit par **tuiles en mémoire partagée** ; somme où chaque thread additionne plusieurs éléments (boucle à pas de grille) avant la réduction en mémoire partagée ; mesure des transferts séparément.